



London's diesel pollution

Researchers say the air pollution crisis in London is choking residents and its time the government do something. <http://dgit.in/PolluteBrit>



Airtel is offering 1000GB

Airtel is offering bonus data of 1000GB for one year to new broadband users who signed up on or after May 16 in NCR <http://dgit.in/1000Airtel>

```
character = character
+ 3;
else /* character is
in [X-Z|x-z] */
character = character
- 23;
}
print(character);
}
```

And voila! You have now already learnt 3 ways to write algorithms (Natural Language, Flow Charts, and Pseudocode). These are some of the most commonly used methods, but there are quite a few other ways of representing an algorithm as well, like drakon-charts or control tables.

TIME COMPLEXITY AND THE DREADED BIG O !

NO. We are not going to explain the mathematical model. We'll give a brief and hopefully understandable explanation. A function or an operations Big-O notation is determined by how much slower is it if the size of inputs is varied.

For example, if you are the messenger sent by Caesar to deliver our cipher message to Anthony and you do not know him by face or his ranks, then instead of looking at one particular tent at the army camp, you may have to visit and search every tent in the camp. Therefore the time required for you to find Mark Anthony, when represented via a mathematical function is commonly denoted by the Big-O notation. Different algorithms try to reduce this time so that even for large inputs (like the size of the Roman army!) the required time of the operation is optimal. If you look for Anthony by visiting one tent after the other, the best case is that you find him in the 1st one and the worst case is that you don't find him at all (mathematically known as the lower and upper bounds respectively).

While programming in most languages, in general you will observe that the time complexity of a function/method increases n times every time you have nested iterations. So if you have one 'for' loop iterating over a size

of n inputs then the time complexity is of the order ' n ' while adding another outer loop with the same no of inputs will raise it to the order of ' n ' * ' n '. So in short if the first loop was taking 5 seconds to complete, then adding an outer loop will increase it to somewhere around 25 second roughly. A simpler explanation of the same can be found here: <http://dgit.in/BigONtn>.

AND FINALLY A FUNNY SORT ALGORITHM THAT TAKES A LOT OF TIME!

The last algorithm that we are going to see today is called BOGOSORT. Note, this algorithm is meant as a programming example and is used mostly as a joke. So, do not implement this in real life programs unless you want your programs to become really, really slow as the Bogosort algorithm has a general time complexity of $O(n!.n)$.

Bogo sort is a highly ineffective algorithm that takes an input array and sorts the array by generating random permutations until the array is sorted. After every iteration, the algorithm reshuffles the entire array and generates a new and completely random sequence and checks if the array is sorted. If not, it repeats itself. The pseudo code for Bogosort (also known as slow-sort, permutation sort, stupid sort, monkey sort etc) is as follows:

```
while not InOrder(list)
do
Shuffle(list)
Done
```

We have removed the implementation details for the shuffle part of this algorithm for simplicity.

Now, go and have fun with algorithms. Find some new ones or play with the old ones. An interesting exercise that you can undertake is to look at existing algorithms and try to figure out their time complexity using the Big-O notation. And once you master that, maybe go looking for a better way to encrypt Caesar's messages or, if you're ambitious, better Google's search algorithm so that it can find a time machine for you! 🕒

*POINTERS

>>Interesting developer videos



*Google I/O 2017 Keynote

The entire developer keynote from this year's Google I/O, different from the main keynote, where Google directly updates Android developers about its plans.

<http://dgit.in/I02017Dev>



*WWDC 2017 - What to expect from Apple

WWDC 2017 Hype - New iOS 11 Beta, macOS 10.13, New iPad Pro 2 & More Updates from Apple! What To Expect at The Keynote to this year's developer conference.

<http://dgit.in/WWDC2017Apple>



*MMX Hill Climb

In this session to hear some of Alphabet's top machine learning experts discuss their cutting-edge research and the opportunities they see ahead.

<http://dgit.in/PPFAIML>